TP2 : Complexité

Algorithmique et Programmation 3
LI 2 Semestre 3 — Université d'Artois

Septembre 2026

Quand c'est possible, pour chacune des questions, précisez la complexité de votre algorithme. Essayez de chercher la meilleur complexité pour répondre aux questions.

Exercice 1 (Somme simple)

Écrire la fonction `somme_factorielles` permettant de calculer la somme suivante :

$$\text{somme-factorielles}(n) = \sum_{k=0}^{n} k!$$

Exercice 2 (Répétitions)

Écrire la fonction `premiere_repetition`, permettant de trouver le premier élément qui se répète dans une liste d'entiers.

```
>>> l = [1, 2, 3, 5, 7, 5, 2, 12, 1]
>>> premiere_repetition(l)
5
```

Exercice 3 (Diagonales de matrices)

Étang donné la matrice bidimensionnelle, écrivez la fonction `affiche_diagonales` qui permet d'afficher toutes les diagonales de la matrice.

Exemple : Pour la matrice suivante :

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix}$$

Le résultat est le suivant :

```
>>>mat = [[1, 2, 3, 4],[5,6,7, 8],[9,10,11,12],[13,14,15,16]]
>>>affiche_diagonales}(mat)
1
5 2
9 6 3
13 10 7 4
14 11 8
15 12
16
```

Exercice 4 (Recherche d'intrus)

Étant donné une liste d'entiers dont chacun apparaît deux fois excepté un qui apparaît une seule fois. Écrire une fonction permettant de trouver ce nombre. Donnez sa complexité et proposer un algorithme de complexité linéaire.

```
>>> l = [1, 2, 3, 5, 7, 5, 2, 1, 3]
>>> intrus(l)
7
```

Exercice 5 (Recherche d'intervalles)

Étant donné une liste d'entiers non triés, écrire la fonction `intervalle_est_dans_liste` permettant de tester si la liste contient tous les entiers d'un certain intervalle. Donnez sa complexité et proposer un algorithme de complexité linéaire.

```
>>> l = [4, 1, 2, 3, 5, 7, 5, 2, 1, 3, 6, 9]
>>> intervalle_est_dans_liste(l, 1, 12)
False
>>> intervalle_est_dans_liste(l, 2, 5)
True
```

Exercice 6 (Recherche de Triplets)

Étant donnée une liste d'entiers l. Écrire une fonction `triplet_avec_ordre` permettant de trouver une instance de i, j, k tel que $0 \leq i < j < k < n$ et $l[i] < l[j] < l[k]$.

```
>>> l = [4, 0, 7, 3, 5, 7, 5, 2, 1, 3, 6, 9]
>>> triplet_avec_ordre(l)
(1, 3, 4)
>>> l = [14, 12, 7, 5, 5, 3, 0]
>>> triplet_avec_ordre(l)
(-1, -1, -1)
```

Exercice 7 (Sous-liste de somme maximale)

Étant donnée une liste d'entiers relatifs (pouvant contenir des nombres négatifs), écrire une fonction `somme_maximale` qui renvoie la plus grande somme possible obtenue par une sous-liste contiguë.

```
>>> l = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
>>> somme_maximale(l)
6   # Correspond a la sous-liste [4, -1, 2, 1]
```

Exercice 8 (Matrice en spirale)

Étant donnée une matrice bidimensionnelle de taille $M \times N$, écrire la fonction `affiche_spirale` qui affiche l'ensemble de ses éléments en parcourant la matrice dans le sens des aiguilles d'une montre (de l'extérieur vers l'intérieur). Préciser la complexité temporelle et spatiale.

```
>>> M = [
...   [1,  2,  3,  4],
...   [5,  6,  7,  8],
...   [9, 10, 11, 12]
... ]
>>> affiche_spirale(M)
1 2 3 4 8 12 11 10 9 5 6 7
```

Exercice 9 (Déplacement des zeros)

Étant donnée une liste d'entiers, écrire une fonction `deplacer_zeros` qui déplace tous les 0 à la fin de la liste tout en conservant l'ordre relatif des autres éléments non nuls. L'opération doit être effectuée **en place** (sans créer de nouvelle liste) avec une complexité temporelle $\Theta(n)$ et une complexité spatiale $\Theta(1)$.

```
>>> l = [0, 1, 0, 3, 12]
>>> deplacer_zeros(l)
>>> print(l)
[1, 3, 12, 0, 0]
```